

Algorithms and Complexity

Contents

1	Asymptotic notation	2
1.1	Relationships	2
1.2	Running time	2
2	The knapsack problem	2
2.1	Greedy strategy	3
2.1.1	Greedy strategy for the fractional problem	3
2.1.2	Greedy strategy for the 0-1 problem	4
3	Greedy algorithms for (some) optimisation problems	5
4	Divide-and-conquer algorithms	5
5	Cographs	5
5.1	Definition	5
5.2	Alternative definition	5
5.3	Cotree	6
5.3.1	Colouring cographs	6
6	The maximum-subarray problem	6
7	The master theorem	7
8	Matrix multiplication	7
8.1	Divide-and-conquer approach	7
8.1.1	Time complexity	8
8.2	Strassen's algorithm	8
9	Rod cutting problem	8
9.1	Brute force	8
9.2	Optimal substructure	9
9.3	Recursive top-down implementation	9
9.4	Dynamic programming	9
9.4.1	Top-down implementation with memoization	10
9.4.2	Bottom-up implementation	10
10	Matrix chain multiplication	11
10.1	Applying dynamic programming	12

10.1.1	Structure of an optimal solution	12
10.1.2	Recursive definition of the value of an optimal solution	12
10.1.3	Computing optimal costs	13
10.1.4	Constructing an optimal solution	14
11	Longest common subsequence	14
11.1	Brute-force	14
11.2	Dynamic programming	14
11.2.1	Optimal substructure	14
11.2.2	Recursive solution	15
11.2.3	Computing the length of an LCS	15
11.2.4	Constructing an LCS	16
12	Matching in graphs	16
12.1	Alternating paths and cycles	17
12.1.1	Running time	19
13	Dijkstra's Algorithm	19
13.1	Adjacency matrix	19
13.2	Shortest path	20
13.3	Single-source shortest paths problem	20
13.4	Relaxing an edge	20
13.5	Dijkstra's algorithm	20
13.6	Properties of shortest paths and relaxation	20
14	Flow networks	21
14.1	Maximum-flow problem	21
14.1.1	Flow	21
14.1.2	Implicit summation	22
14.2	The Ford-Fulkerson method	22
14.2.1	Residual networks	22
14.2.2	Augmenting path	22
14.2.3	Cuts	22
14.2.4	Basic Ford-Fulkerson algorithm	23
15	Decision problems	24
15.1	Defining a decision problem	24
15.1.1	Reachability	24
15.1.2	Encoding scheme	24
15.2	Languages	24
15.3	Complexity measures	25
15.3.1	Dynamic complexity measures	25
15.4	Independent Set and Vertex Cover	25
15.5	Clique	26
16	The complexity class P	27
16.1	Time complexity	27
16.2	Time complexity classes	27
16.3	The class P	27

16.3.1	Different models of computation	27
16.3.2	Encodings	28
16.4	Proving a problem is in P	28
16.5	CNF	28
16.5.1	Satisfiability	29
16.6	Polynomial-time reduction	30
16.6.1	k -colourability	30
16.6.2	Reducing 2-Colourability to 2-Satisfiability	31
17	The complexity class NP	31
17.1	Certificate	31
17.2	Verifiers	31
17.3	The class NP	32
17.4	Non-deterministic machines	32
17.4.1	Non-deterministic TM time complexity	33
17.4.2	The class NTIME(f)	33
17.5	Alternative definition of NP	33
17.6	P = NP	33
17.7	Complete problems	34
17.7.1	Ordering by reduction	34
17.7.2	Complete problems	34
17.8	NP-completeness	34
17.8.1	NP-completeness of Satisfiability	35
17.9	Ladner's theorem	35
17.10	NP-intermediate candidates	35
17.11	NP-hard	35
17.12	Proving reducability	36
17.13	NP-completeness of Vertex Cover	36
17.14	NP-completeness of Clique	37
17.15	NP-completeness of Hitting Set	38
17.15.1	NP-completeness of 3-SAT	38
17.16	Reducing SAT to Clique	38

1 Asymptotic notation

For two given functions $f(n) \geq 0$ and $g(n) \geq 0$:

- $f(n) = O(g(n))$ if $\exists c > 0, n_0$ such that $f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$
- $f(n) = \Omega(g(n))$ if $\exists c > 0, n_0$ such that $f(n) \geq c \cdot g(n) \quad \forall n \geq n_0$
- $f(n) = \Theta(g(n))$ if $\exists c_1, c_2 > 0, n_0$ such that $c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall n \geq n_0$
- $f(n) = o(g(n))$ if $\forall c_1 > 0, \exists n_0$ such that $f(n) \leq c_1 \cdot g(n) \quad \forall n \geq n_0$
- $f(n) = \omega(g(n))$ if $\forall c_2 > 0, \exists n_0$ such that $f(n) \geq c_2 \cdot g(n) \quad \forall n \geq n_0$

1.1 Relationships

For any $f(n), g(n)$: 1. $f(n) = \Theta(g(n)) \iff g(n) = \Theta(f(n))$ 2. $f(n) = \Theta(g(n)) \iff f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$ 3. $f(n) = O(g(n)) \iff g(n) = \Omega(f(n))$ 4. $f(n) = o(g(n)) \iff g(n) = \omega(f(n))$ 5. $f(n) = O(g(n)) \iff$ either $f(n) = \Theta(g(n))$ or $f(n) = o(g(n))$ 6. $f(n) = \Omega(g(n)) \iff$ either $f(n) = \Theta(g(n))$ or $f(n) = \omega(g(n))$

1.2 Running time

The running time $T(I)$ of an algorithm α on input I is the time taken by α on I until α halts. Inputs are classified according to their size.

The worst-case running time $T(n)$ of algorithm α on inputs of size n is defined as $T(n) := \max\{T(I) : I \text{ has size } n\}$

In calculating the running time of an algorithm, the RAM model of computation is used: - every primitive operation takes constant time, typically considered to be '1'

Let the pseudocode of algorithm α consist of p lines, where the execution of each line i requires time 1.

Let q_i be the number of times line i is executed on input I .

The running time of α on input I is

$$T(I) := \sum_{i=1}^p q_i$$

2 The knapsack problem

There are two variants of the knapsack problem: - The 0 - 1 knapsack problem (the 'classic' variant): - A thief robs a store and finds n items. - Item i is worth v_i euros and weighs w_i kilos. - The thief can carry at most W kilos in knapsack. - Which items can he take to maximise his profit? - The fractional knapsack problem: - same setup, but this time the thief can take fractions of items

It can be assumed that the knapsack is not big enough to carry all items, i.e., $W < w_1 + \dots + w_n$, otherwise the thief would just take all items.

2.1 Greedy strategy

2.1.1 Greedy strategy for the fractional problem

The fractional knapsack problem can be easily solved using a greedy strategy: 1. Compute the value per kilo, $r_i = v_i/w_i$ for each item. 2. Sort items by value per kilo in decreasing order. 3. Take as much as possible of item with greatest ‘value per kilo’, then take as much as possible of item with the next greatest ‘value per kilo’, etc., until the total weight equals the knapsack capacity W .

For example, take $W = 10\text{kg}$ and

$$\begin{aligned}v_1 &= \text{£}4, & w_1 &= 4\text{ kg} \Rightarrow r_1 = 1, \\v_2 &= \text{£}10, & w_2 &= 2\text{ kg} \Rightarrow r_2 = 5, \\v_3 &= \text{£}12, & w_3 &= 8\text{ kg} \Rightarrow r_3 = 1.5, \\v_4 &= \text{£}20, & w_4 &= 5\text{ kg} \Rightarrow r_4 = 4.\end{aligned}$$

Then, the greedy strategy takes: 1. all of item 2 (total 2 kilos), 2. all of item 4 (total 5 kilos), 3. 3 kilos of item 3,

for a total value of $10 + 20 + 3 \cdot 1.5 = 34.5$.

Running time The running time of the greedy strategy can be determined by looking at the running time of each of the steps: - Step 1 (computing r_i for each i) can be done in $\theta(n)$ - Step 2 (sorting the r_i array) can be done in $\theta(n \log n)$ - Step 3 (selecting the items) can be done in $\theta(n)$

Therefore, the worst case running time is $\theta(n \log n)$.

Corectness We have that the n items are sorted by $r_i = v_i/w_i$ in decreasing order, i.e.,

$$r_1 \geq r_2 \geq \dots \geq r_n$$

Two items p and q with $r_p = r_q$ are considered to be equivalent. Hence, we may assume that:

$$r_1 > r_2 > \dots > r_n$$

The total available amount of item i is w_i . Let k be such that

$$w_1 + \dots + w_k \leq W \quad \text{and} \quad w_1 + \dots + w_{k+1} > W$$

That is, k is the last item that can be taken completely before the knapsack capacity is exceeded. Such a k exists as we assumed the knapsack was not able to take all items.

Let $x = (x_1, \dots, x_n)$ denote a solution, where $0 \leq x_i \leq w_i$ is the amount the thief takes of item i for $i = 1, \dots, n$.

The total value stolen is then $\sum_{i=1}^n r_i x_i$.

The greedy solution x is given by

$$\begin{aligned}x_i &= w_i & \text{for } 1 \leq i \leq k, \\x_{k+1} &= W - (w_1 + \dots + w_k), \\x_i &= 0 & \text{for } k+2 \leq i \leq n.\end{aligned}$$

Let $x^* = (x_1^*, \dots, x_n^*)$ be an optimal solution.

Note that we must have $x_1^* + \dots + x_n^* = W$.

Suppose $x^* \neq x$. Then: - $x_i^* < x_i$ for some $i \leq k+1$ - let $j := \min\{i : x_i^* < x_i\}$ - j is the first item that less of is taken in the optimal solution - also, $x_\ell^* > x_\ell$ for some $\ell > j$ - ℓ is the first item that more of is taken in the optimal solution

Define a new solution $y = (y_1, \dots, y_n)$ by

$$y = \begin{cases} y_j = \min\{w_j, x_j^* + (x_\ell^* - x_\ell)\}, \\ y_\ell = x_\ell^* - (y_j - x_j^*) \\ y_i = x_i^* \quad \text{otherwise.} \end{cases}$$

This moves as much weight as possible from item ℓ to item j : - the min ensures we don't exceed the available amount w_j - the other items remain unchanged

The total weight remains the same, W , but the value increases:

$$\sum_{i=1}^n r_i y_i - \sum_{i=1}^n r_i x_i^* = (r_j - r_\ell)(y_j - x_j^*) > 0,$$

This is a contradiction: x^* cannot be optimal if we can improve it, hence $x^* = x$

2.1.2 Greedy strategy for the 0-1 problem

The same greedy algorithm does not work for the 0-1 knapsack problem.

For instance, take $W = 6\text{kg}$, and:

$$\begin{aligned} v_1 &= \text{£}3, & w_1 &= 4\text{kg} \Rightarrow r_1 = 3/4, \\ v_2 &= \text{£}2, & w_2 &= 3\text{kg} \Rightarrow r_2 = 2/3, \\ v_3 &= \text{£}2, & w_3 &= 3\text{kg} \Rightarrow r_3 = 2/3. \end{aligned}$$

The optimal solution is to steal objects 2 and 3 for a total of £4.

However, the greedy strategy will steal object 1 for a total of £3.

A special case of 0-1 problem Suppose that, in a 0-1 knapsack instance, the orders of the items: - when sorted by decreasing value v_i , and - when sorted by increasing weight w_i

are the same (i.e. $v_1 \geq v_2 \geq \dots \geq v_n$ and $w_1 \leq w_2 \leq \dots \leq w_n$).

This means the first item is both the most valuable and the lightest, the second is the second-most valuable and slightly heavier, etc.

In this case, applying the greedy algorithm to the problem will produce the optimal solution.

3 Greedy algorithms for (some) optimisation problems

In an optimisation problem, we are looking for a solution that optimises (i.e. maximises / minimises) some value: - in such a problem every solution has a value - an optimal solution is a solution with optimal (minimum or maximum) value. There can be many optimal solutions - the aim is to find any one of them - any algorithm for an optimisation problem goes through a sequence of steps - at each step there is a set of choices, from which one must be chosen - a greedy algorithm makes in every step the choice that looks to be the best at the moment - this is called a greedy choice or an locally optimal choice - not all optimisation problems can be solved by greedy algorithms (e.g. the 0-1 knapsack problem)

4 Divide-and-conquer algorithms

Many useful algorithms are recursive, i.e. call themselves recursively to deal with subproblems (e.g. merge sort, quick sort).

At each level of recursion, the algorithm: - divides the problem into a number of subproblems - conquers the subproblems by solving them: recursively, or directly if the subproblem sizes are small enough - combine the solutions for the subproblems into a solution for the original problem

The running times of divide-and-conquer algorithms are naturally given by recurrence relations, which express the running time on a problem of size n in terms of the running times on smaller subproblems plus the time spent dividing the problem and combining the results. For example, for merge sort:

$$T(n) = \begin{cases} \Theta(1) & \text{if } n = 1, \\ 2T\left(\frac{n}{2}\right) + \Theta(n) & \text{if } n > 1. \end{cases}$$

5 Cographs

5.1 Definition

Let G_1 and G_2 be two vertex-disjoint graphs.

The disjoint union $G_1 + G_2$ is the graph with: - vertex set $V(G_1) \cup V(G_2)$ and - edge set $E(G_1) \cup E(G_2)$

The join $G_1 \times G_2$ is the graph with: - vertex set $V(G_1) \cup V(G_2)$ and - edge set $E(G_1) \cup E(G_2) \cup E^*$, where E^* is the set of all edges between vertices of G_1 and vertices of G_2

A graph G is a cograph if it can be created using these rules: 1. the 1-vertex graph is a cograph 2. if G_1 and G_2 are cographs, then so is $G_1 + G_2$ 3. if G_1 and G_2 are cographs, then so is $G_1 \times G_2$

5.2 Alternative definition

Let P_4 denote the path on 4 vertices: - a graph G contains P_4 as an induced subgraph if G can be modified to P_4 via vertex deletions - if not, then G is said to be P_4 -free.

A graph G is a cograph $\iff G$ is P_4 -free.

5.3 Cotree

Let G be a cograph. The cotree T_G of G is the unique decomposition tree that satisfies: 1. The root r of the tree represents the graph $G_r = G$ 2. Every leaf x of the tree represents exactly one vertex of G , and vice versa 3. Every internal node x of the tree has at least two children, it is either labeled $+$ or $\times$, and it represents an induced subgraph G_x of G , defined as follows: - if x is a $+$ -node, then G_x is the disjoint union of all graphs G_y where y is a child of x - if x is a $\times$ -node, then G_x is the join of all graphs G_y where y is a child of x 4. Along the path from the root r to any leaf, the labels of the internal nodes alternate between $+$ and $\times$

Any cograph G has a unique cotree. However, if we drop condition 4., there may be many trees representing G .

An example is shown below.

There is an algorithm for determining the cotree of a cograph, but it will not be included here.

5.3.1 Colouring cographs

A colouring of a graph is an assignment of colors to the vertices of the graph such that any two adjacent vertices are coloured differently.

A colouring using at most k colors is a k -colouring.

The chromatic number, χ_G of a graph G is the smallest integer k for which G has a k -colouring.

In general, this is complex, with no known polynomial-time algorithm. However, for a cograph G , its cotree T_G can be used to find its chromatic number. A bottom-up approach is followed - only process node x after first processing its children: - leaves: each leaf represents a single vertex of G . The chromatic number of a single-vertex graph is 1. - union-nodes: if x is a $+$ -node, then χ_{G_x} is the maximum of the chromatic numbers of the graphs G_y , where y is a child of x - join-nodes: if x is a $\times$ -node, then χ_{G_x} is the sum of the chromatic numbers of the graphs G_y , where y is a child of x .

This way, the last node processed is the root r , which represents the graph: $G_r = G$.

As the number of nodes of T is $O(n)$, and $O(1)$ time is spent per node, the total running time is polynomial.

6 The maximum-subarray problem

The maximum-subarray problem involves finding the maximum sum of a contiguous subarray of an array A .

The brute force approach - checking all pairs of start and end indices - is $\theta(n^2)$.

It is possible to do better, using divide-and-conquer. The idea is as follows: - split the array in half at the midpoint mid - any maximum subarray falls into one of three cases: - entirely in the left half, $A[\text{low} \dots \text{mid}]$ - entirely in the right half, $A[\text{mid} \dots \text{high}]$ - crossing the midpoint mid - the first two cases can be solved recursively - to solve the third case: - compute the max sum ending at mid from the left - compute the max sum starting at $\text{mid} + 1$ from the right - add the two sums together

The running time of this is $T(n) = \theta(n \log n)$.

7 The master theorem

The master theorem is used to solve recurrence relations, and is defined as follows.

Let $f(n)$ be a function on the natural numbers, and let $T(n)$ be defined by

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

for some constants $a \geq 1$ and $b > 1$. Then:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \cdot \log n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $a \cdot f(n/b) \leq c \cdot f(n)$ for some constant $c < 1$ and for sufficiently large n , then $T(n) = \Theta(f(n))$.

8 Matrix multiplication

The problem we are interested in is multiplying two square matrices, $A, B \in \mathbb{R}^{n \times n}$ to get their product, $C \in \mathbb{R}^{n \times n}$.

Using the definition of matrix multiplication, $C_{ij} = \sum_{k=1}^n a_{ik}b_{kj}$, it can be performed in $\theta(n^3)$: - there are n^2 elements - for each element, there are n multiplications, and $n-1$ additions; $n+(n-1) = \theta(n)$

8.1 Divide-and-conquer approach

Suppose we have two $n \times n$ matrices A and B . For simplicity, assume n is a power of 2 (so we can evenly split the matrices).

Each matrix is partitioned into 4 equal-sized blocks, called block matrices:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}, \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}, \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

Here: - Each A_{ij}, B_{ij}, C_{ij} is a $\frac{n}{2} \times \frac{n}{2}$ matrix - For example, A_{11} is the top-left quadrant of A

Then:

$$\begin{aligned} C_{11} &= A_{11}B_{11} + A_{12}B_{21}, \\ C_{12} &= A_{11}B_{12} + A_{12}B_{22}, \\ C_{21} &= A_{21}B_{11} + A_{22}B_{21}, \\ C_{22} &= A_{21}B_{12} + A_{22}B_{22} \end{aligned}$$

Each of the smaller matrices can then be multiplied recursively.

The pseudocode for such an algorithm can be as follows:

MULT(A, B)

```
let C be a new $n \times n$ matrix
if n = 1 then
```

```

    c11 <- a11b11
else
  partition A, B, and C into the block matrices
  C11 <- MULT(A11, B11) + MULT(A12, B21)
  C12 <- MULT(A11, B12) + MULT(A12, B22)
  C21 <- MULT(A21, B11) + MULT(A22, B21)
  C22 <- MULT(A21, B12) + MULT(A22, B22)
return C

```

8.1.1 Time complexity

The running time of this approach satisfies $T(n) = 8T(\frac{n}{2}) + \theta(n^2)$, since there are: - 8 multiplications of $\frac{n}{2} \times \frac{n}{2}$ matrices, and - 4 matrix additions, each taking time $\theta(\frac{n}{2} \cdot \frac{n}{2}) = \theta(n^2)$

Solving using the master theorem show that $T(n) = \theta(n^3)$, so this is not an improvement over using the definition.

8.2 Strassen's algorithm

Strassen's algorithm builds on this divide-and-conquer approach: 1. Divide A , B , and C into four parts as before - running time: $\theta(1)$ 2. Create 10 auxiliary matrices $S_1, \dots, S_{10} \in \mathbb{R}^{n/2 \times n/2}$, obtained by sums or differences of $A_{11}, \dots, B_{22}$. - running time: $\theta(n^2)$ 3. Using all the matrices available, recursively compute seven product matrices $P_1, \dots, P_7 \in \mathbb{R}^{n/2 \times n/2}$ - running time: $7T(n/2)$ 4. Compute $C_{11}, \dots, C_{22}$ by adding/subtracting combinations of $P_1, \dots, P_7$. - running time: $\theta(n^2)$

The running time of this approach satisfies $T(n) = 7T(\frac{n}{2}) + \theta(n^2)$

Using the master theorem, we obtain:

$$T(n) = \theta(n^{\log_2 7}) = \theta(n^{2.8074})$$

9 Rod cutting problem

The rod cutting problem is as follows.

Given: - A rod of length n . - A price list p_i for $i = 1, \dots, n$ where p_i is the selling price of a rod of length i .

Determine the maximum revenue r_n obtained by cutting up the rod and selling the pieces.

9.1 Brute force

One way to solve this problem is by brute force - trying all possible integer combinations (we assume the rod must be cut into integer-length parts).

An integer partition of a positive integer n is a list of positive integers $a_1, \dots, a_k$ such that $a_1 \leq a_2 \leq \dots \leq a_k$ and $\sum_{i=1}^k a_i = n$.

The number of integer partitions of n increases exponentially with n .

Therefore, the brute force method has an exponential running time, and is impractical.

9.2 Optimal substructure

As a general solution,

$$r_n = \max\{p_i + r_{n-i} : 1 \leq i \leq n\}$$

Essentially, the maximum revenue from a rod of length n is obtained by choosing the best first cut length i , earning p_i from that piece, and then optimally cutting the remaining rod of length $n - i$.

Therefore, to solve the original problem of size n , we solve independent smaller problems of the same type, but of smaller sizes.

Optimal substructure means that an optimal solution to a problem can be constructed from optimal solutions to its subproblems (which may be solved independently).

9.3 Recursive top-down implementation

An algorithm that relies on optimal substructure is given in pseudocode below:

```
ROD(p, n)

if n == 0 then
    return 0
q <- -1
for i <- 1 to n do
    q <- max{q, p[i] + ROD(p, n - i)}
return q
```

This algorithm: - tries every possible first cut - solves the remaining subproblem recursively - then chooses the best option

However, this algorithm recomputes the same values many times, and therefore the time complexity is exponential.

9.4 Dynamic programming

Rather than re-solving the same subproblem repeatedly, the subproblems can be arranged so that each subproblem is solved only once, with its solution stored. If the solution is needed thereafter, it is retrieved instead of recomputed.

Dynamic programming implements this idea by using additional memory to save computation time. It is therefore an example of time-memory trade off.

A dynamic programming algorithm runs in polynomial time when both: - the number of distinct subproblems involved is polynomial - each subproblem can be solved in polynomial time

Note that for any algorithm that uses subproblems, the subproblems have to be independent. Two subproblems are independent if the solution to one subproblem does not affect the solution to another subproblem of the same problem.

Dynamic programming is effective because it utilises both: - optimal substructure (where the solution to a problem can be constructed from optimal solutions to its subproblems), and - overlapping subproblems (where the same subproblem appears in multiple larger problems)

It uses memoization, which is the process of caching the results of function calls (subproblem solutions) so that repeated calls with the same arguments can return the cached result instead of recomputing it.

9.4.1 Top-down implementation with memoization

The top-down implementation from earlier can be improved using memoization.

The procedure is still written recursively in a natural manner, but modified to save the result of each subproblem (in an array).

Then, when the procedure needs to solve a subproblem, it checks if it has previously solved this subproblem; if not, the procedure computes the value in the usual manner.

We say the recursive procedure has been memoized: it remembers what results it has computed previously.

The pseudocode for this is given below.

```
MEMOIZED-ROD(p, n)
Let r[0..n] be a new array
for i ← 0 to n do
    r[i] ← -1
return MEMOIZED-ROD-AUX(p, n, r)

MEMOIZED-ROD-AUX(p, n, r)
if r[n] ≥ 0 then
    return r[n]    // r[n] has been previously computed
if n == 0 then
    q ← 0    // the returned value r[0]
else
    q ← -1
    for i ← 1 to n do    // compute r[n]
        q ← max{q, p[i] + MEMOIZED-ROD-AUX(p, n-i, r)}
r[n] ← q
return q
```

9.4.2 Bottom-up implementation

Another approach to implement dynamic programming is the bottom-up method.

It depends on the idea of the ‘size’ of a subproblem, such that solving any particular subproblem relies on solving smaller subproblems.

The subproblems are sorted by size and solved in size order, smallest first, with the solutions saved.

Each subproblem is solved only once, and when it is first seen, all of its prerequisite subproblems have already been solved.

The pseudocode for this is given below:

```
BOTTOM-UP-ROD(p, n)
Let r[0..n] be a new array
```

```

r[0] <- 0
for j <- 1 to n do
  q <- -1
  for i <- 1 to j do
    q <- max{q, p[i] + r[j - i]} // r[j - i] is known
  r[j] <- q
return r[n]

```

Running time For rod cutting, both the top-down and bottom-up dynamic programming approaches have a worst case running time of $\theta(n^2)$.

This is easy to see for the bottom-up approach, since it contains a (single) nested for loop.

The top-down approach solves each subproblem for sizes $0, 1, \dots, n$ once. Therefore, to solve a problem of size n : - the for loop iterates n times - the total number of iterations is the sum $1 + 2 + \dots + n$, an arithmetic series, giving $\theta(n^2)$

For other problems, both approaches typically have the same complexity, but: - the bottom-up approach has lower constants, due to less overhead for procedure calls - top-down may sometimes avoid solving certain subproblems entirely, if they are never needed in the recursion tree. This can make it faster in practice for some sparse problems

10 Matrix chain multiplication

Let A be a $p \times q$ matrix, and B be a $q' \times r$ matrix, i.e.,

$$A = \begin{pmatrix} a_{1,1} & a_{1,2} & \dots & a_{1,q} \\ a_{2,1} & a_{2,2} & \dots & a_{2,q} \\ \vdots & \vdots & \ddots & \vdots \\ a_{p,1} & a_{p,2} & \dots & a_{p,q} \end{pmatrix}, \quad B = \begin{pmatrix} b_{1,1} & b_{1,2} & \dots & b_{1,r} \\ b_{2,1} & b_{2,2} & \dots & b_{2,r} \\ \vdots & \vdots & \ddots & \vdots \\ b_{q,1} & b_{q,2} & \dots & b_{q,r} \end{pmatrix}$$

The product $C = AB$: - is not defined if $q \neq q'$ - is the $p \times r$ matrix if $q = q'$, where

$$c_{i,j} = \sum_{k=1}^q a_{i,k} b_{k,j}$$

This matrix has $p \cdot r$ entries, and for each entry, there are q multiplication and $q - 1$ additions = $\theta(q)$ operations per entry. - the runtime of this matrix multiplication is $\theta(pqr)$

Matrix multiplication is associative: - the multiplication ordering does not change the product - e.g, $A_1 A_2 A_3 = (A_1 (A_2 A_3)) = ((A_1 A_2) A_3)$

Even though the result is the same, one order of multiplication may involve significantly fewer multiplications than another.

For example, take three matrices:

$$A_1 : 10 \times 100, \quad A_2 : 100 \times 5, \quad A_3 : 5 \times 50$$

There are two ways of parenthesizing: - $(A_1 A_2) A_3$ requires - $(10 \times 100 \times 5) + (10 \times 5 \times 50) = 7,500$ multiplications - $A_1 (A_2 A_3)$ requires - $(100 \times 5 \times 50) + (10 \times 100 \times 50) = 75,000$ multiplications

The matrix chain multiplication problem is defined as follows.

Given a sequence (chain) $\langle A_1, \dots, A_n \rangle$ of n matrices, where A_i has dimensions $p_{i-1} \times p_i$, determine the optimal parenthesization of the product $A_1 A_2 \dots A_n$ that minimises the total number of scalar multiplications.

A product of matrices

$$A_1 A_2 \dots A_N$$

is fully parenthesized if it is: - either a single matrix, - or the product of two fully parenthesized matrix products, surrounded by parentheses

Let $P(n)$ denote the number of ways to parenthesize a product of n matrices.

If the product is fully parenthesized, then: - it is the product of two fully parenthesized sub-products - the split of these two sub-products occurs between the k th and $(k+1)$ st matrices, where $1 \leq k \leq n-1$

This provides the recurrence:

$$P(n) = \begin{cases} 1, & \text{if } n = 1 \\ \sum_{k=1}^{n-1} P(k) P(n-k), & \text{if } n \geq 2 \end{cases}$$

Using a brute-force method to check every possible parenthesization has an exponential running time.

10.1 Applying dynamic programming

To use dynamic programming, follow a four-step sequence: 1. Characterise the structure of an optimal solution. 2. Recursively define the value of an optimal solution. 3. Compute the value of an optimal solution. 4. Construct an optimal solution from computed information.

10.1.1 Structure of an optimal solution

To optimally parenthesize $A_i \dots A_j$, suppose that we split the product between A_k and A_{k+1} : - both parenthesizations of $A_i \dots A_k$ and $A_{k+1} \dots A_j$ must be optimal

The cost of the optimal solution for $A_i \dots A_j$ is the sum of: - the cost to optimally parenthesize $A_i \dots A_k$ - the cost to optimally parenthesize $A_{k+1} \dots A_j$ - the cost of multiplying them together

10.1.2 Recursive definition of the value of an optimal solution

Let $m[i, j]$ be the minimum number of multiplications required to compute $A_i \dots A_j$.

The goal is to compute $m[1, n]$.

$m[1, n]$ can be computed recursively, for $i \leq j$:

$$m[i, j] = \begin{cases} 0, & \text{if } i = j, \\ \min_{i \leq k < j} \{m[i, k] + m[k+1, j] + p_{i-1} p_k p_j\}, & \text{if } i < j. \end{cases}$$

There are two cases: - the base case: if $i = j$, then $m[i, j] = 0$, since a single matrix needs no multiplication - the recursive case: if $i < j$, then: - check all possible places to split - use the minimum of all of these

10.1.3 Computing optimal costs

There are only a few distinct subproblems: - one for every pair of i and j , so there are $\theta(n^2)$ subproblems - there are many overlapping subproblems

A bottom-up approach can be used: - computing $m[i, j]$ for a product of $j - i + 1$ matrices only uses values for products of fewer than $j - i + 1$ matrices. - order matrices by their chain length $\ell = j - i + 1$ - this is just the number of matrices being multiplied in the subproblem - store the answers in a table $m[1 \dots n, 1 \dots n]$ - starting from the shortest chain length (the smallest number of matrices in the product), calculate $m[i, j]$ - first fill $m[i, i] = 0$ - then compute chains of length 2, then length 3 etc. until length n - for each chain, try every possible split and choose the cheapest

Pseudocode for this is given below:

```
MATRIX-CHAIN-ORDER(p)
Let m[1..n, 1..n] be a new table
for i <- 1 to n do
    m[i, i] <- 0
for l <- 2 to n do
    for i <- 1 to n - l + 1 do
        j <- i + l - 1
        m[i, j] <- inf
        for k <- i to j - 1 do
            m[i, j] <- min{m[i, j], m[i, k] + m[k + 1, j] + pi-1pkpj}
return m
```

So far, we have determined the minimum number of multiplications, but not where the parenthesis should be placed to achieve it.

However, it is easy to modify the solution to keep track of where to place the parenthesis: - for every i and j , it is enough to remember the cut point k , where we cut the product $A_i \dots A_j$ into $A_i \dots A_k$ and $A_{k+1} \dots A_j$ - an additional array $s[i, j]$ can be used to keep track of all those k s

```
MATRIX-CHAIN-ORDER(p)
Let m[1..n, 1..n] and s[1..n - 1, 2..n] be new tables
for i <- 1 to n do
    m[i, i] <- 0
for l <- 2 to n do
    for i <- 1 to n - l + 1 do
        j <- i + l - 1
        m[i, j] <- inf
        for k <- i to j - 1 do
            q <- min{m[i, j], m[i, k] + m[k + 1, j] + pi-1pkpj}
            if q < m[i, j] then
                m[i, j] <- q
                s[i, j] <- k
return m
```

10.1.4 Constructing an optimal solution

The table $s[1..n-1, 2..n]$ hold the information to print out the right parenthesization.

```
Algorithm PRINT-PAR(s, i, j)
if i == j then
    print "Ai"
else
    print "("
    PRINT-PAR(s, i, s[i, j])
    PRINT-PAR(s, s[i, j] + 1, j)
    print ")"
```

This part has running time $O(n)$.

Running time The main algorithm contains three nested for loops, therefore the running time of the algorithm is $O(n^3)$.

It can further be shown that the running time is actually $\theta(n^3)$.

11 Longest common subsequence

The longest common subsequence problem is as follows.

Given two sequences, find the longest sequence of characters (longest common subsequence) that:

- appears in both sequences
- appears in the same order
- is not necessarily contiguous (i.e., characters don't have to be next to each other)

For example, for the strings **S1** and **S2** below, **S3** is the longest common subsequence:

Formally, given a sequence $X = \langle x_1, \dots, x_m \rangle$, another sequence $Z = \langle z_1, \dots, z_k \rangle$ is a subsequence of X if there exist indices $i_1 < i_2 < \dots < i_k$ such that

$z_1 = x_{i_1}, z_2 = x_{i_2}, \dots, z_k = x_{i_k}$. - the elements do not have to be consecutive — only the order must be preserved

A common subsequence of X and Y is a subsequence of both X and Y .

11.1 Brute-force

A possible brute-force approach is to:

- enumerate all subsequences Z of X
- check whether Z is also a subsequence of Y
- keep track of the longest subsequence that appears in both

If X has m indicies, there are 2^m subsets - therefore, the brute force approach has exponential running time

11.2 Dynamic programming

Let $X = \langle x_1, x_2, \dots, x_n \rangle$, and let the i th prefix of X be $X_i = \langle x_1, x_2, \dots, x_i \rangle$.

11.2.1 Optimal substructure

Let $Z = \langle z_1, \dots, z_k \rangle$ be an LCS of $X = \langle x_1, \dots, x_m \rangle$ and $Y = \langle y_1, \dots, y_n \rangle$.

1. If $x_m = y_n$, then:

- $z_k = x_m = y_n$
 - Z_{k-1} is an LCS of X_{m-1} and Y_{n-1}
2. If $x_m \neq y_n$, then:
- $z_k \neq x_m$ implies that Z is an LCS of X_{m-1} and Y
3. If $x_m \neq y_n$, then:
- $z_k \neq y_n$ implies that Z is an LCS of X and Y_{n-1}

In words, this means: - look at the last character of X and Y - if the last characters match: - this character must be the last character in the LCS, because if it weren't used, we could append it and get a longer common subsequence (contradiction) - everything before it, Z_{k-1} , must be an LCS of the prefixes X_{m-1} and Y_{n-1} - if the last characters don't match: - at least one of them is not part of the LCS - if the LCS does not end with the last character of X , then discarding this character X_m does not affect the LCS - therefore, Z is an LCS of X_{m-1} and Y - if the LCS does not end with the last character of Y , then discarding this character Y_n does not affect the LCS - therefore, Z is an LCS of X and Y_{n-1}

Therefore, there is an optimal substructure: - an LCS of X and Y contains within it an LCS of some prefixes of X and Y

One or two subproblems have to be examined each time: 1. if $x_m = y_n$ we must find an LCS of X_{m-1} and Y_{n-1} 2. if $x_m \neq y_n$ we have to find: - an LCS of X_{m-1} and Y - an LCS of X and Y_{n-1} - keep whichever is longer

There is also the overlapping subproblems property: - we may need to find LCS's of $\{X_{m-1}, Y\}$ and of $\{X, Y_{n-1}\}$ - each of these subproblems has the subproblem of finding an LCS of $\{X_{m-1}, Y_{n-1}\}$ - therefore, many other subproblems share common subproblems

11.2.2 Recursive solution

Let $c[i, j]$ be the length of an LCS of the prefixes X_i and Y_j .

Therefore,

$$c[i, j] = \begin{cases} 0 & \text{if } i = 0 \text{ or } j = 0, \\ c[i - 1, j - 1] + 1 & \text{if } i, j > 0 \text{ and } x_i = y_j, \\ \max\{c[i, j - 1], c[i - 1, j]\} & \text{if } i, j > 0 \text{ and } x_i \neq y_j. \end{cases}$$

Unlike the rod cutting and the matrix chain multiplication problems, here we rule out some subproblems based on the conditions of the problem (i.e. $x_i = y_j$ or $x_i \neq y_j$).

Used directly, this recursive equation gives an exponential time recursive algorithm. However: - every recursive call is of the form $c[i, j]$ - so a subproblem is identified by the pair (i, j) , where $i \in \{0, 1, \dots, m\}$ and $j \in \{0, 1, \dots, n\}$ - so the number of distinct subproblems = $(m+1)(n+1) = \theta(mn)$

Therefore, using dynamic programming to only solve each subproblem once and store the result for reuse if needed, reduces the algorithm to polynomial time, more specifically, $\theta(mn)$.

11.2.3 Computing the length of an LCS

A bottom-up approach can be used: - create a length table $c[0 \dots m, 0 \dots n]$ - $c[i, j]$ is length of the LCS of prefixes X_i and Y_j - create a direction table $b[1 \dots m, 1 \dots n]$ - $b[i, j]$ is a pointer that tells us how the optimal value in $c[i, j]$ was obtained - this table is used later to reconstruct the actual

LCS - initialise the base cases: - for all i , set $c[i, 0] = 0$ - for all j , set $c[0, j] = 0$ - fill the tables in row by row - once the table is complete, the length of the LCS is stored in: $c[m, n]$

Pseudocode for this is given below:

```

LCS(X, Y)
Let b[1..m, 1..n] and c[0..m, 0..n] be new tables
for i <- 0 to m do c[i, 0] <- 0
for j <- 0 to n do c[0, j] <- 0
for i <- 1 to m do
    for j <- 1 to n do
        if xi == yj then
            c[i, j] <- c[i - 1, j - 1] + 1
            b[i, j] <- "UL" // upper-left subproblem
        else if c[i - 1, j] >= c[i, j - 1] then
            c[i, j] <- c[i - 1, j]
            b[i, j] <- "U" // upper subproblem
        else
            c[i, j] <- c[i, j - 1]
            b[i, j] <- "<-" // left subproblem
return c and b

```

11.2.4 Constructing an LCS

To recover the actual subsequence (not just its length): - start at cell (m, n) - follow the pointers in b : - whenever we see a diagonal pointer (UL): - output the corresponding character - move to $(i - 1, j - 1)$ - if the pointer is up (U), move to $(i - 1, j)$ - if the pointer is left (<-), move to $(i, j - 1)$ - stop when you reach the top row or leftmost column - the characters collected (in reverse order) form the LCS

```

PRINT-LCS(b, X, i, j)
if i == 0 or j == 0 then
    return ""
if b[i, j] == "UL" then
    PRINT-LCS(b, X, i - 1, j - 1)
    print xi
else if b[i, j] == "U" then
    PRINT-LCS(b, X, i - 1, j)
else
    PRINT-LCS(b, X, i, j - 1)

```

Outputting the LCS is $O(m + n)$, as the total number of steps is at most $m + n$.

An example, combining both tables, is given below:

12 Matching in graphs

Let $G = (V, E)$ be an undirected graph.

When modeling a real-world application, usually: - vertices $v \in V$ represent some objects of interest, e.g. cities on a map - edges $e \in E$ represent relations between pairs of vertices, e.g. roads between

cities

However, often the edges become the interesting objects to study - an edge unites or matches two vertices

A matching $M \subseteq E$ in G is an independent (non-conflicting) set of edges, i.e. a set of edges where no two edges share a common vertex. No object is used twice; there are no conflicts, as each object is matched with at most one partner.

Matchings are useful as many real-world problems are about making non-conflicting pairings. - for example, suppose there is a graph where each vertex represents a football team, and each edge represents a potential game between two teams - if a number of football games are to be scheduled at the same time, a matching would be required, as a team can only play one other team at a time

A bipartite graph is a graph where the vertices can be split into two disjoint groups, such that all edges go between the two groups, and never within the same group. Such a graph can be two-coloured.

Formally, a graph $G = (V, E)$ is bipartite if its vertex set V can be partitioned into two sets, $V = X \cup Y$ such that - $X \cap Y = \emptyset$ (they don't overlap) - every edge connects a vertex in X to a vertex in Y - no edge connects two vertices both in X - no edge connects two vertices both in Y

Many assignment problems can be formulated as finding a special matching in a bipartite graph $G = (X \cup Y, E)$. - for example, vertices in X are workers, and vertices in Y are projects to be assigned to workers; each edge represents a job a worker is able to do

A matching M is called: - maximal, if M is not a proper subset of any other matching M' in G - maximum, if no other matching M' in G contains a strictly greater number of edges - perfect, if every vertex of G is an endpoint of an edge in M

The matching number ν_G of G is the size of a maximum matching in G .

Note: - Every maximum matching is also maximal - A maximal matching is not necessarily maximum - A perfect matching is always maximum

12.1 Alternating paths and cycles

Let $G = (V, E)$ be a graph and M be a matching in G .

Let $P = (v_0, v_1, \dots, v_k)$ be a simple path with edges $e_1 = v_0v_1, e_2 = v_1v_2, \dots, e_k = v_{k-1}v_k$.

Then: - P is an M -alternating path if the edges $e_1, e_2, \dots, e_k$ are alternately in M and in $E \setminus M$.

- P is an M -augmenting path if P is M -alternating and neither v_0 nor v_k is covered by M - this requires $M = \{e_2, e_4, \dots, e_{k-1}\}$ - therefore, k is odd in an augmenting path

Similarly, a cycle C in G is an M -alternating cycle if its edges alternately belong to M and $E \setminus M$.

A vertex u is matched by M if it is the end-vertex of an edge in M ; otherwise u is unmatched by M .

For two sets A and B , the symmetric difference is the set $A \oplus B = (A \setminus B) \cup (B \setminus A)$. - for example: Let $A = \{1, 2, 3, 4\}$ and $B = \{3, 4, 5, 6\}$

Then:

$$A \setminus B = \{1, 2\}$$

$B \setminus A = \{5, 6\}$
so $A \otimes B = \{1, 2, 5, 6\}$

Lemma Let G be a graph with a matching M and an M -alternating path $P = (v_0, v_1, \dots, v_k)$. If each of the endpoints v_0, v_k of P is: - either unmatched by M ,
- or matched by $M \cap P$ (matched using edges inside the path)

then $M \otimes P$ is another matching in G .

Corollary If P is an M -augmenting path, then in the new matching $M \otimes P$:

- $|M \otimes P| = |M| + 1$,
- all the matched vertices in M remain matched,
- two additional vertices v_0, v_k become matched in $M \otimes P$.
– this is called an augmentation of M : replacing M with $M \otimes P$.

Lemma Let $G = (V, E)$ be an undirected graph, and let M and M^* be two matchings in G . Then the subgraph $H = (V, M \otimes M^*)$ of G is the disjoint union of: - isolated vertices, - alternating cycles with respect to both M and M^* , - alternating paths with respect to both M and M^* .

Lemma Let $G = (V, E)$ be a graph. The following are equivalent: 1. M is a maximum matching
2. G has no M -augmenting path

Proof First, it must be proved that if M is a maximum matching, then there is no M -augmenting path: - Let G be a graph with a maximum matching M and an M -augmenting path P . - Then $|M \otimes P| = |M| + 1$, i.e. the matching $M \otimes P$ has more edges than M . - hence, M is not a maximum matching - this is a contradiction, therefore, if M is a maximum matching, then there is no M -augmenting path

Next, it must be proved that if there exists no M -augmenting path P , then M is a maximum matching: - Suppose M is not a maximum matching - Let M' be a maximum matching - The subgraph $H = (V, M \otimes M')$ contains only alternating paths and cycles (with respect to M and M') - Each alternating cycle C has an even number of edges - therefore, C has the same number of edges in M and M' - Since $|M'| > |M|$, there must be an odd path P in H , in which the first and last edges belong to M' - P is an M -augmenting path - this is a contradiction, therefore, M is a maximum matching

Corollary Let $G = (V, E)$ be a graph, and let M and M^* be two matchings in G such that $|M^*| = |M| + k$ for some $k \geq 1$. Then G has at least k pairwise vertex-disjoint M -augmenting paths.

Proof

- The subgraph $H = (V, M \otimes M^*)$ consists of alternating paths and alternating cycles with respect to both M and M^* (and possibly isolated vertices).
- The symmetric difference $M \otimes M^*$ contains exactly k more edges from M^* than from M .
- Therefore, H contains k pairwise vertex-disjoint odd-length paths that start and end with edges of M^* .
- These paths are M -augmenting paths.

The main takeaway from the lemmas above is that if a matching is not maximum, then there exists an augmenting path, and if you flip along that path, the matching gets bigger by 1.

Therefore, there is an algorithmic method for solving the matching problem: 1. Let $M = \emptyset$ 2. Check if there is an M -augmenting path P 3. If P exists, then $M := M \oplus P$ and go to Step 2. Otherwise output M .

This requires an algorithm to determine if there is an augmenting path P : - A vertex u is M -reachable from v if there is an M -alternating path from v to u : - therefore, use a search algorithm to find all M -reachable vertices from v - if we find an unmatched M -reachable vertex u then there is an M -augmenting path from v to u - otherwise, there is no M -augmenting path from v , so try other vertices

The idea for the search algorithm is: - grow a search tree T rooted at v - each path from v to any vertex u in the tree is alternating - every vertex that is visited is labeled odd/even - if vertex u is odd/even, the path from v in T is odd/even - if an unmatched vertex is labeled odd, an augmenting path has been found

Note that this only works for bipartite graphs, since they do not have odd-cycles.

The algorithm can then work as follows: - The algorithm maintains a LIST of labeled vertices. Initially, $LIST = \{v\}$, $label(v) = \text{"even"}$. - It iteratively removes a vertex from LIST and continues searching. - To examine an even vertex u : - Scan its neighbors $N(u)$. - If $w \in N(u)$ is unlabeled: - assign $label(w) = \text{"odd"}$, - add w to LIST. - If w is unmatched, then an augmenting path has been found and the algorithm can stop - To examine an odd vertex u (which has been matched): - Explore the unique edge $ux \in M$. - If x is unlabeled: - assign $label(x) = \text{"even"}$, - add x to LIST.

The algorithm terminates if: - either LIST is empty
- there is no augmenting path from v ,
- or we assign the label "odd" to an unmatched vertex
- an augmenting path has been found.

12.1.1 Running time

Since we only explore each edge once, the algorithm for finding an augmenting path runs in $O(|E|)$ time.

Each occasion results in a matching that has one more edge than the previous matching. The largest possible matching has $\frac{|V|}{2}$ vertices, so the augmenting path algorithm is run at most $\frac{|V|}{2}$ times.

Therefore, the total running time of the algorithm for finding a maximum matching in a bipartite graph is $O(|E||V|)$ time.

13 Dijkstra's Algorithm

13.1 Adjacency matrix

An adjacency matrix can be used to represent a graph with edge weights: - a square matrix is used
- rows and columns are labelled with the vertices

The entry in row i and column j is: - w if there is an edge of weight w from i to j , - 0 if there is no edge.

13.2 Shortest path

The weight (or length) of a path from a vertex u to a vertex v is the sum of the weights of the edges in the path.

Note that a weight may be negative.

The shortest path (i.e. with least weight) between u and v is denoted $\delta(u, v)$. If there is no path, then $\delta(u, v) = \infty$.

13.3 Single-source shortest paths problem

The single-source shortest paths problem involves finding the shortest path from a specific source vertex s to all other vertices in the graph.

13.4 Relaxing an edge

Relaxing an edge means checking whether going through that edge gives a shorter path to a vertex, and updating the distance if it does.

13.5 Dijkstra's algorithm

1. Initialisation
 - the node A is the source node
 - the set S stores the vertices v for which $\delta(A, v)$ has been found (initially empty)
 - the set Q stores all the other vertices (initially contains all vertices)
 - $d(v)$ is an estimate of $\delta(A, v)$
 - $d(A) = 0$ (initially infinite for all other vertices)
2. While Q is not empty:
 - remove from Q the vertex u for which $d(u)$ is minimum
 - add this vertex u to S
 - relax all edges leaving u

The running time of Dijkstra's algorithm is $O(V^2)$.

13.6 Properties of shortest paths and relaxation

- Triangle Inequality
 - For all edges (u, v) , we have $\delta(s, v) \leq \delta(s, u) + w(u, v)$.
- Optimal Substructure
 - Any subpath of a shortest path is also a shortest path.
- Upper Bound Property
 - For every vertex v , we always have $d(v) \geq \delta(s, v)$.
- No-path Property
 - If $\delta(s, v) = \infty$, then we have $d(v) = \infty$ at every iteration.
- Convergence Property
 - If there is a shortest path from s to v including the edge (u, v) and if $d(u) = \delta(s, u)$, then we obtain $d(v) = \delta(s, v)$ when (u, v) is relaxed.

14 Flow networks

Take a scenario where: - material is transferred in a network from a “source” to a “sink” - source produces material at a steady rate, sink consumes at same rate

For example, water through a system of pipes or current through an electrical network.

Edges have a given capacity - the maximum material transfer per unit of time

Vertices (other than source/sink) are “junctions”: - material flows through them without collecting in them - entering rate = exiting rate (flow conservation)

14.1 Maximum-flow problem

The maximum-flow problem involves computing the greatest possible rate of transportation from source to sink in a flow network.

A flow network is defined by a directed graph $G = (V, E)$, with: - two distinguished vertices: source s and sink t - each edge $(u, v) \in E$ has non-negative capacity $c(u, v) \geq 0$ - if $(u, v) \notin E$, we assume $c(u, v) = 0$.

We assume that every vertex lies on some path from s to t : - for each $v \in V$, there is a path $s \rightsquigarrow v \rightsquigarrow t$ - therefore, G is connected, so $|E| \geq |V| - 1$

14.1.1 Flow

A flow in G is a real-valued function $f : V \times V \rightarrow \mathbb{R}$ that satisfies three properties: - Capacity constraint: - For all $u, v \in V$, we require $f(u, v) \leq c(u, v)$ - Flow from one vertex to another must not exceed given capacity. - Skew symmetry: - For all $u, v \in V$, we require $f(u, v) = -f(v, u)$ - Flow from vertex u to vertex v is negative of flow in reverse direction. - Flow conservation: For all $u \in V - s, t$, we require $\sum_{v \in V} f(u, v) = 0$ - Total flow out of a vertex (other than source or sink) is 0.

$f(u, v)$ is called the flow from u to v . It can be positive, zero, or negative.

Total flow The total positive flow entering vertex v is

$$\sum_{u \in V: f(u, v) > 0} f(u, v)$$

The total positive flow leaving vertex u is

$$\sum_{v \in V: f(u, v) > 0} f(u, v)$$

The total net flow at a vertex v is: - total positive flow leaving v - total positive flow entering v

Total net flow is: - positive at source - negative at sink - 0 at all other vertices

Flow value The value of flow f is defined as the total flow leaving the source (and thus entering the sink):

$$|f| = \sum_{v \in V} f(s, v)$$

14.1.2 Implicit summation

Let $X, Y \subseteq V$. Then:

$$f(X, Y) = \sum_{x \in X} \sum_{y \in Y} f(x, y)$$

Commonly occurring identities: - For all $X \subseteq V$, we have $f(X, X) = 0$ - Because each $f(u, v)$ and $f(v, u) = -f(u, v)$ cancel each other. - For all $X, Y \subseteq V$, we have $f(X, Y) = -f(Y, X)$ - Generalization of $f(X, X) = 0$, same reasoning. - For all $X, Y, Z \subseteq V$ with $X \cap Y = \emptyset$, we have - $f(X \cup Y, Z) = f(X, Z) + f(Y, Z)$ - $f(Z, X \cup Y) = f(Z, X) + f(Z, Y)$ - Split summation into two: one over X , one over Y

14.2 The Ford-Fulkerson method

The Ford-Fulkerson method is an iterative method for solving the maximum flow problem: 1. Start with $f(u, v) = 0$ for all $u, v \in V$ 2. At each iteration, increase flow value by finding an augmenting path — a path from source to sink along which we can increase flow — and then augment flow along this path. 3. Repeat until no augmenting path can be found

14.2.1 Residual networks

The residual network consists of edges that can admit more flow.

Consider vertices u and v . The amount of additional flow we can push from u to v before exceeding capacity $c(u, v)$ is the residual capacity of (u, v) : - $c_f(u, v) = c(u, v) - f(u, v)$

Note: When flow $f(u, v)$ is negative, then residual capacity $c_f(u, v)$ is greater than $c(u, v)$.

Given flow network $G = (V, E)$ and flow f , the residual network of G induced by f is $G_f = (V, E_f)$ with

$$E_f = \{(u, v) \in V \times V : c_f(u, v) > 0\}$$

i.e. each residual edge that can admit flow that is strictly positive.

Let $G = (V, E)$ be a flow network, f be a flow in G , G_f be the residual network of G induced by f , and let f' be a flow in G_f . Then the flow sum $f + f'$ with

$$(f + f')(u, v) = f(u, v) + f'(u, v)$$

is a flow in G with value $|f + f'| = |f| + |f'|$.

14.2.2 Augmenting path

An augmenting path is a path in the residual graph from the source s to the sink t where every edge has residual capacity > 0 .

Along this path, you can increase the flow by the minimum residual capacity on the path.

14.2.3 Cuts

A cut (S, T) is a partition of all the nodes V into two sets: - S — contains the source s - T — contains the sink t

Every node is in exactly one of the two sets: $S \cup T = V$, $S \cap T = \emptyset$

If f is a flow in G , then $f(S, T)$ is the net flow across cut (S, T) ; its capacity is $c(S, T)$. - net flow includes negative flows across the cut; capacity includes only non-negative values

The net flow across any cut is the same. - since flow is conserved at every node apart from s and t

A minimum cut is a cut with minimum capacity over all cuts.

The value of the flow f in a network G is upper bounded by the capacity of any cut (S, T) in G .

Max-flow min-cut If f is a flow in a flow network $G = (V, E)$ with source s and sink t , then the following conditions are equivalent: 1. f is a maximum flow in G . 2. The residual network G_f contains no augmenting path. 3. $|f| = c(S, T)$ for some cut (S, T) of G .

14.2.4 Basic Ford-Fulkerson algorithm

Input: G, s, t

```

for each edge  $(u, v)$  in  $E$  do
     $f(u, v) \leftarrow 0$ 
     $f(v, u) \leftarrow 0$ 
end for

```

```

while there exists a path  $P = s \rightarrow t$  in the residual network  $G_f$  do
    {i.e. augmenting path in  $G$ }
     $c_f(P) \leftarrow \min\{c_f(u, v) : (u, v) \text{ in } P\}$ 

    for each edge  $(u, v)$  in  $P$  do
         $f(u, v) \leftarrow f(u, v) + c_f(P)$ 
         $f(v, u) \leftarrow -f(u, v)$ 
    end for
end while

```

- Start with zero flow
 - Set flow on every edge to 0
- While there is a path from s to t (augmenting path)
 - Find a path where more flow can be sent (in the residual graph)
 - Find the bottleneck capacity
 - * Look at all edges in the path
 - * Take the minimum remaining capacity
 - Push flow along the path
 - * Add the bottleneck value to each forward edge
 - * Update backward edges to allow flow to be undone if needed
- Repeat
 - Keep finding new paths and pushing flow
- Stop when no augmenting path exists
 - No more flow can be sent from s to t
- Result
 - The total flow is the maximum flow

Running time The running time of the algorithm is highly dependent on how augmenting paths are determined for the while loop.

If chosen poorly: - value of flow increases with each iteration - but possibly too slowly - in extreme cases it might never terminate, and it can even not converge to the value of maximum flow (this can happen only if the capacities are irrational numbers)

15 Decision problems

A decision problem is a problem whose answer is simply yes or no.

An optimisation problem is a problem that asks for the best solution among all feasible solutions.

Every optimisation problem has a decision problem counterpart:

- For example, the shortest path problem:
 - optimisation: given vertices u and v , what is the shortest path from u to v .
 - decision: given vertices u, v and value k does there exist a path from u to v with length at most k ?

An optimisation problem has a polynomial algorithm if and only if the corresponding decision problem has a polynomial algorithm.

15.1 Defining a decision problem

A decision problem is usually defined by:

- describing a generic instance of the problem
- asking a yes/no question about that instance

15.1.1 Reachability

The reachability problem is an example of a decision problem:

- Given a finite directed graph G and vertices s and t , is there a path in G from s to t

15.1.2 Encoding scheme

In order to input a problem to a computer, each problem instance must be encoded as a string of symbols over some alphabet (typically binary).

Therefore, an encoding scheme is needed.

This encoding scheme must be concise and efficient:

- numbers should be represented efficiently (e.g. binary rather than unary/base 1)
- unnecessary or redundant information should be avoided

This is important because the running time of algorithms is measured in terms of the input size. An inefficient encoding could artificially change the apparent complexity of a problem.

In complexity theory, reasonable encoding schemes differ only polynomially in size, so they do not change the fundamental classification of a problem (for example, whether a problem is in P or NP).

15.2 Languages

An alphabet Σ is a finite set of symbols - e.g., $\Sigma = 0, 1$

A string over Σ is a finite sequence of symbols from Σ - e.g., 01101

A language over Σ is any set of strings over Σ .

For a decision problem Π , and an encoding scheme e over some alphabet Σ

$$L(\Pi, e)$$

is defined to be the set of all encoded instances for which the answer to Π is yes.

This set is called the language associated with Π and e .

Thus, solving a decision problem is equivalent to deciding whether a given encoded instance belongs to the language $L(\Pi, e)$.

15.3 Complexity measures

Every decidable problem has at least one algorithm that solves it.

To classify problems, we study properties of the set of all algorithms that solve the problem.

A key idea is to consider the efficiency of the best possible algorithm in this set, measured in terms of resource usage. This is a dynamic complexity measure.

A complexity measure is a function that quantifies the amount of computational resources used by an algorithm as a function of the input.

15.3.1 Dynamic complexity measures

The most important computational resources are usually time and space.

Using Turing Machines as the model of computation:

- The time complexity of a Turing Machine T is the function $Time_T(x)$ which is the number of computation steps taken by T on input x .
- The space complexity of a Turing Machine T is the function $Space_T(x)$ which is the number of tape cells visited during the computation of T on input x .

If $T(x)$ does not halt, then both $Time_T(x)$ and $Space_T(x)$ are undefined.

15.4 Independent Set and Vertex Cover

The independent set problem is defined as:

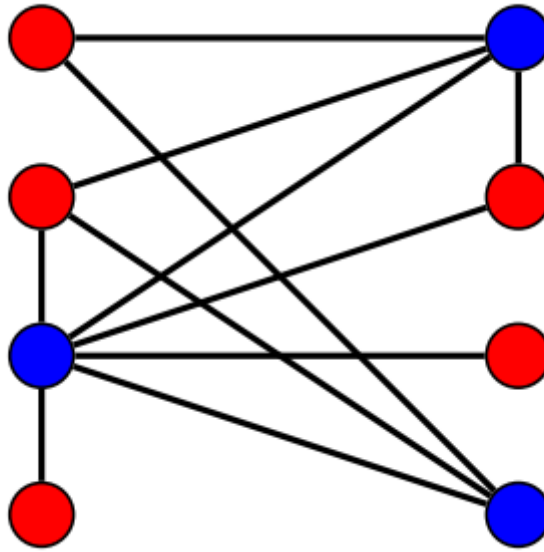
- Given a graph $G = (V, E)$ and an integer k ,
- does G contain an independent set of size at least k ?

An independent set is a set of vertices such that no two vertices in the set are joined by an edge.

The vertex cover problem is defined as:

- Given a graph $G = (V, E)$ and an integer k ,
- does G contain a vertex cover of size at most k ?

A vertex cover is a set of vertices such that every edge in the graph has at least one endpoint in the set.



In this example graph:

- there is an independent set of size 5 (the red vertices)
- there is a vertex cover of size 3 (the blue vertices)

For a graph G with n vertices:

- G has an independent set of size k if and only if G has a vertex cover of size $n - k$

Therefore, the independent set and vertex cover decision problems are polynomial-time equivalent (they can be transformed into each other in polynomial time via the above relationship).

15.5 Clique

The clique problem is defined as:

- Given a graph $G = (V, E)$ and an integer k ,
- does G contain a clique of size at least k ?

A clique is a set of vertices in which every pair of distinct vertices is connected by an edge.

The complement of a graph $G = (V, E)$, denoted $\overline{G}$, is the graph with:

- the same vertex set V
- an edge between two vertices u and v in G if and only if $(u, v) \notin E$

A graph G has an independent set of size k if and only if its complement $\overline{G}$ has a clique of size k .

Therefore, the Clique problem and the Independent Set problem are polynomial-time equivalent, since one can be transformed into the other via taking graph complements in polynomial time.

16 The complexity class **P**

16.1 Time complexity

For any function f , a decidable language L has time complexity $O(f)$ if there exist:

- a deterministic Turing machine T that decides L , and
- constants $c > 0$ and $n_0 \geq 0$,

such that for every input x with $|x| \geq n_0$,

$$\text{Time}_T(X) \leq c \cdot f(|x|)$$

Here:

- $|x|$ denotes the length of the input,
- $\text{Time}_T(x)$ is the number of computation steps taken by T on input x .

16.2 Time complexity classes

For a function f , the complexity class

$$\text{TIME}(f)$$

(or sometimes $\text{DTIME}(f)$, for deterministic time) is the set of all languages L for which there exists a deterministic Turing machine that decides L in $O(f)$ time.

16.3 The class **P**

The complexity class **P** is defined by:

$$\mathbf{P} = \bigcup_{k \geq 0} \text{TIME}[n^k]$$

This is the class of all languages that can be decided in polynomial time by a deterministic Turing machine.

The class **P** is commonly considered a mathematical model of tractable (efficiently solvable) problems.

However, the correspondence with “practical efficiency” is only approximate:

- A polynomial-time algorithm with a large exponent may still be impractical for real input sizes
- The hidden constant factors in the $O()$ notation may also be very large and dominate runtime in practice

Generally for naturally occurring problems in computer science, the degree and constants are not too large.

16.3.1 Different models of computation

- A k -tape Turing machine that runs in t steps can be simulated by an equivalent single-tape Turing machine in $O(t^2)$ time.

- A single-tape Turing machine running in t steps can be simulated by a k -tape Turing machine in $O(t)$ time.

Therefore, the complexity class **P** is invariant under these standard models of computation (up to polynomial factors), including single-tape, multi-tape, and other reasonable deterministic machine models.

16.3.2 Encodings

For any integer n , the length of its representation in base b_1 base b_2 differ by at most a constant factor, provided $b_1, b_2 \geq 2$.

For any graph G , the length of the encoding of G :

- as an adjacency matrix, and
- as a list of edges

differ in size by at most a polynomial factor in the number of vertices.

Hence the class **P** is invariant under these reasonable encodings of inputs (up to polynomial transformations of input length).

16.4 Proving a problem is in **P**

The class **P** is considered robust:

- it is invariant under reasonable choices of computational model (e.g. single-tape vs multi-tape Turing machines)
- it is also invariant under reasonable encodings of inputs

Therefore, when proving that a problem is in **P**, the exact machine model or low-level encoding details are usually not important.

To show that a problem (language) is in **P**, the standard approach is:

- Provide a deterministic algorithm that solves the problem in polynomial time

This algorithm can be described at a high level (e.g. in pseudo-code), as long as its correctness and polynomial running time can be justified:

- even a straightforward polynomial-time algorithm is useful, because it establishes that the problem is efficiently solvable in theory
- however, finding such an algorithm often requires identifying a strategy that is significantly better than brute-force search

16.5 CNF

Many important computational problems involve logical formulas over Boolean variables.

A Boolean formula f is in conjunctive normal form (CNF) if it can be written as:

$$f = C_1 \wedge \dots \wedge C_m$$

where each C_i is a clause.

A clause is a disjunction (OR) of literals:

$$C_i = L_{i1} \vee \dots \vee L_{ik}$$

A literal is either: - a variable, e.g., x - or its negation, e.g., $\neg x$

A formula is in k -CNF if every clause contains at most k literals.

16.5.1 Satisfiability

A Boolean formula f in CNF is satisfiable if there exists an assignment of TRUE / FALSE values to its variables such that the formula evaluates to TRUE.

- f is TRUE iff all clauses are TRUE
- a clause is TRUE iff at least one of its literals is TRUE

The Satisfiability problem The Satisfiability problem (SAT) is:

Given a CNF formula f , does there exist a truth assignment to its variables such that f evaluates to TRUE?

k -Satisfiability problem A restricted version is the k -SAT problem:

Given a CNF formula in which every clause has at most k literals, is the formula satisfiable?

2-Satisfiability 2-Satisfiability is in P.

The proof is as follows:

0. Declare:
 - all clauses unsatisfied
 - all literals unassigned
1. Select an arbitrary unassigned variable x . Assign:
 - $x = \text{TRUE}$
 - $\neg x = \text{FALSE}$
2. Select an unsatisfied clause $I_i \vee I_j$:
 - if both literals are unassigned:
 - ignore the clause
 - if at least one literal is assigned TRUE:
 - declare the clause satisfied
 - if one literal is FALSE and the other is unassigned:
 - assign the unassigned literal to TRUE
 - assign its negation to FALSE
 - mark the clause as satisfied
 - if both literals are FALSE:
 - restart the algorithm
 - set $x = \text{FALSE}$ and $\neg x = \text{TRUE}$
 - if the same conflict occurs again after this restart, declare the formula unsatisfiable
3. Repeat step 2 until either:
 - all clauses are satisfied:
 - return satisfiable

- remaining unsatisfied clauses contain only unassigned variables:
 - return to step 1

16.6 Polynomial-time reduction

An alternative way to prove that a problem is in $\mathbf{P}$ is by using a reduction.

Informally, a problem P is reducible to a problem Q if an algorithm for solving Q can be used as a subroutine to solve P .

Formally: a language L_1 is polynomially reducible to L_2 , denoted

$$L_1 \leq L_2$$

if there exists a function f such that:

- f is computable in polynomial time, and
- for all inputs x :

$$x \in L_1 \iff f(x) \in L_2$$

The function f transforms instances of problem L_1 into instances of problem L_2 such that:

- YES-instances stay YES-instances
- NO-instances stay NO-instances

So solving L_2 automatically gives a way to solve L_1 .

The key property of this is that:

- if $L_1 \leq L_2$ and $L_2 \in \mathbf{P}$, then $L_1 \in \mathbf{P}$

16.6.1 k -colourability

A (vertex) colouring of a graph $G = (V, E)$ is a function:

$$f : V \rightarrow 1, \dots, k$$

A colouring is proper if for every edge $(u, v) \in E$,

$$f(u) \neq f(v)$$

That is, adjacent vertices must receive different colours.

The k -Colourability problem is:

Given a graph G , does there exist a proper colouring of G using at most k colours?

Note that when $k = 2$, this problem becomes exactly the bipartite graph problem.

Given a graph G , can the vertices be split into two disjoint sets, such that no vertex shares an edge with any other vertex in its set?

16.6.2 Reducing 2-Colourability to 2-Satisfiability

The 2-Colourability (bipartite) problem can be reduced to 2-SAT:

- for each vertex v_i of the graph
 - create a variable x_i
- for each edge (v_i, v_j) :
 - add two clauses:
 - * $(x_i \vee x_j)$
 - * $(\neg x_i \vee \neg x_j)$

This converts the 2-Colourability problem to the 2-Satisfiability problem in $O(V + E)$.

Solving the 2-Colourability problem can now be done by solving the equivalent 2-Satisfiability problem:

- if the formula is satisfiable, define the 2-colouring by setting TRUE variables to colour 1 and FALSE to colour 2
- if two adjacent vertices get the same colour, then the corresponding edge clauses are not satisfied (contradiction)
- thus we obtain a proper 2-colouring

17 The complexity class NP

17.1 Certificate

A certificate (or witness) is a string that provides evidence that an input belongs to a language.

More formally, for a language L , a certificate for an input x is a string c that can be used by a polynomial-time verification algorithm to confirm that:

$$x \in L$$

- A certificate is often thought of as a proposed solution that can be checked efficiently.
- For inputs whose answer is YES, there exists a certificate that verifies this.
- For inputs whose answer is NO, no valid certificate exists.

For example:

- Satisfiability (SAT):
 - a certificate is a satisfying assignment of truth values to the variables
- k -Colourability:
 - a certificate is a proper k -colouring of the graph
- Hamiltonian Cycle:
 - a certificate is a Hamiltonian cycle in the graph

17.2 Verifiers

An acceptor machine V that halts on all inputs is called a verifier for a language L if

$$L = \{w \mid V \text{ accepts } \langle w, c \rangle \text{ for some string } c\}$$

- the string c is called a certificate (or witness) for w
- the verifier receives both the input w and the certificate c
- if $w \in L$, then there exists a certificate c that causes V to accept
- If $w \notin L$, then no certificate causes V to accept

A verifier is said to be polynomial-time if:

- V runs in polynomial time with respect to the size of its input, and
- there exists a polynomial p such that, for every $w \in L$, there is a certificate c satisfying

$$|c| \leq p(|w|)$$

That is, every YES-instance has a certificate whose length is polynomial in the size of the input.

17.3 The class NP

The class **NP** is the set of all languages that have a polynomial-time verifier.

That is, a language is in **NP** if every YES-instance has a polynomial-size certificate that can be verified in polynomial time.

Examples of problems in **NP** include:

Composite Number > Given a positive integer k , are there integers $u, v > 1$ such that $u \cdot v = k$

- A certificate is the pair (u, v) .
- Verification consists of checking that:
 - $u, v > 1$, and
 - $u \cdot v = k$
- Both checks can be performed in polynomial time.

Subset Sum > Given a collection of positive integers $S = a_1, \dots, a_k$ and a target integer t , is there a subset $T \subseteq S$ whose elements sum to t ?

- A certificate is the subset T .
- Verification consists of summing the elements of T and checking whether the result is t .
- This can be done in polynomial time.

Some problems that are probably not in **NP** include:

No Hamiltonian Cycle > Given a graph G , is it true that G has no Hamiltonian cycle?

- a Hamiltonian cycle itself is a certificate that a graph does have a Hamiltonian cycle
- However, it is not known how to provide a polynomial-size certificate that proves a graph has no Hamiltonian cycle.

17.4 Non-deterministic machines

An alternative definition of the class **NP** is given using non-deterministic Turing machines.

A non-deterministic Turing machine NT can be viewed as a computation tree:

- $NT(x)$ denotes the tree of all possible computation paths on input x
- Each node is a configuration, and each branch represents a possible nondeterministic choice

- NT accepts x if at least one computation path ends in an accept state

17.4.1 Non-deterministic TM time complexity

Let NT be a non-deterministic Turing machine.

The time complexity of NT on input x , denoted:

$$\text{NTime}_{NT}(x)$$

is defined as the number of steps in the shortest accepting path of $NT(x)$ if there is one, otherwise it is the number of steps in the shortest rejecting path.

If not all paths of $NT(x)$ halt, then $\text{NTime}_{NT}(x)$ is undefined.

For a function f , we say a language L is decidable in non-deterministic time $O(f)$ if

- there exists a non-deterministic Turing machine NT such that:
 - NT decides L
 - and there exist constants $c > 0$, n_0 such that for all inputs x with $|x| > n$, every computation path of $NT(x)$ halts within:

$$\text{NTime}_{NT}(x) \leq c \cdot f(|x|)$$

17.4.2 The class $\text{NTIME}(f)$

The class

$$\text{NTIME}(f)$$

is the set of all languages that can be decided by a nondeterministic Turing machine in time $O(f)$.

17.5 Alternative definition of NP

Using non-deterministic machines, NP can be defined as:

$$\text{NP} = \bigcup_{k \geq 0} \text{NTIME}[n^k]$$

This is the class of all languages that can be decided by a nondeterministic Turing machine in polynomial time.

17.6 $\text{P} = \text{NP}$

The question of whether $\text{P} = \text{NP}$ is one of the most important open problems in computer science.

It asks whether every problem whose solution can be verified efficiently can also be solved efficiently.

Equivalently, does the existence of a short (polynomial-size) certificate guarantee the existence of a polynomial-time algorithm to find such a certificate?

17.7 Complete problems

Any complexity class can be structured using polynomial-time reductions.

- We say L_1 is related to L_2 if $L_1 \leq L_2$
- This induces a notion of relative difficulty between problems

This allows us to group problems into equivalence classes:

- Two problems are in the same equivalence class if they are polynomial-time reducible to each other

$$L_1 \leq L_2 \text{ and } L_2 \leq L_1$$

- Such problems are considered equally hard (up to polynomial time)

17.7.1 Ordering by reduction

Polynomial-time reductions define a preorder on problems:

- if $L_1 \leq L_2$, then L_1 is no harder than L_2

This induces a partial ordering on equivalence classes of problems.

17.7.2 Complete problems

A problem L in a complexity class C is called complete for C if:

- $L \in C$, and
- every problem in C reduces to L in polynomial time

$$\forall L' \in C, \quad L' \leq L$$

Complete problems are the hardest problems in the class (up to polynomial-time reductions): - If we can solve one complete problem efficiently, then we can solve all problems in the class efficiently

17.8 NP-completeness

NP-complete problems are the hardest problems in NP.

They have the property that:

- they are in NP, and
- every problem in NP can be reduced to them in polynomial time

This means:

- If any NP-complete problem can be solved in polynomial time, then every problem in NP can be solved in polynomial time.

To show that L is NP-complete, it must be shown that every language in NP can be reduced to L in polynomial time:

- once one NP-complete language L_0 is known, any language L can be shown to be NP-complete by showing that $L_0 \leq L$ and $L \in NP$.

17.8.1 NP-completeness of Satisfiability

The Cook-Levin theorem is that $\geq$ Satisfiability is NP-complete

so:

- SAT is in NP, and
- every problem in NP can be reduced to SAT in polynomial time

This would mean that:

- $\mathbf{P} = \mathbf{NP}$ if and only if Satisfiability is in $\mathbf{P}$.

17.9 Ladner's theorem

Ladner's theorem states that:

- if $\mathbf{P} \neq \mathbf{NP}$, then there exist problems in NP that are neither in $\mathbf{P}$ nor NP-complete.

Such problems are called NP-intermediate problems.

This means that, if $\mathbf{P} \neq \mathbf{NP}$, then:

- $\mathbf{NP}$ is strictly larger than $\mathbf{P}$
- not all problems in $\mathbf{NP} \setminus \mathbf{P}$ are NP-complete
- there exist middle problems of intermediate difficulty

17.10 NP-intermediate candidates

Proving that a problem is NP-intermediate would imply that $\mathbf{P} \neq \mathbf{NP}$. Therefore, no problem has been proven to be NP-intermediate.

However, there are several candidates that are believed to lie in NP-intermediate.

A leading candidate is:

Graph Isomorphism

Given two graphs $G = (V_G, E_G)$ and $H = (V_H, E_H)$, are G and H isomorphic?

Two graphs are isomorphic if there is a bijection

$$f : V_G \rightarrow V_H$$

such that

$$(u, v) \in E_G \iff (f(u), f(v)) \in E_H$$

17.11 NP-hard

To prove a problem Π is **NP**-complete, two steps must be performed:

1. Show that $\Pi \in \mathbf{NP}$
2. Find a known **NP**-complete problem Π' and show that $\Pi' \leq \Pi$

If step 2 can be shown, but step 1 is not known or does not hold, then Π is **NP**-hard.

17.12 Proving reducibility

There are three general methods to prove that a known NP-complete problem Π' is polynomial-time reducible to a problem Π :

- restriction
 - show that Π' is a special case (restriction/subproblem) of Π
 - That is, every instance of Π' is also an instance of Π
- local replacement
 - show that every basic unit (e.g. variable, clause, edge) in an instance of Π' can be replaced by a corresponding structure in Π
 - the replacement must be uniform and preserve correctness
- component design
 - show that instances of Π can be used to construct gadgets (components) that simulate the behaviour of Π'
 - these gadgets are then combined to encode any instance of Π'

17.13 NP-completeness of Vertex Cover

Recall the vertex cover problem:

Given a graph $G = (V, E)$ and an integer k , does G contain a vertex cover of size at most k ?

A vertex cover is a set of vertices such that every edge in the graph has at least one endpoint in the set.

This problem is in **NP**:

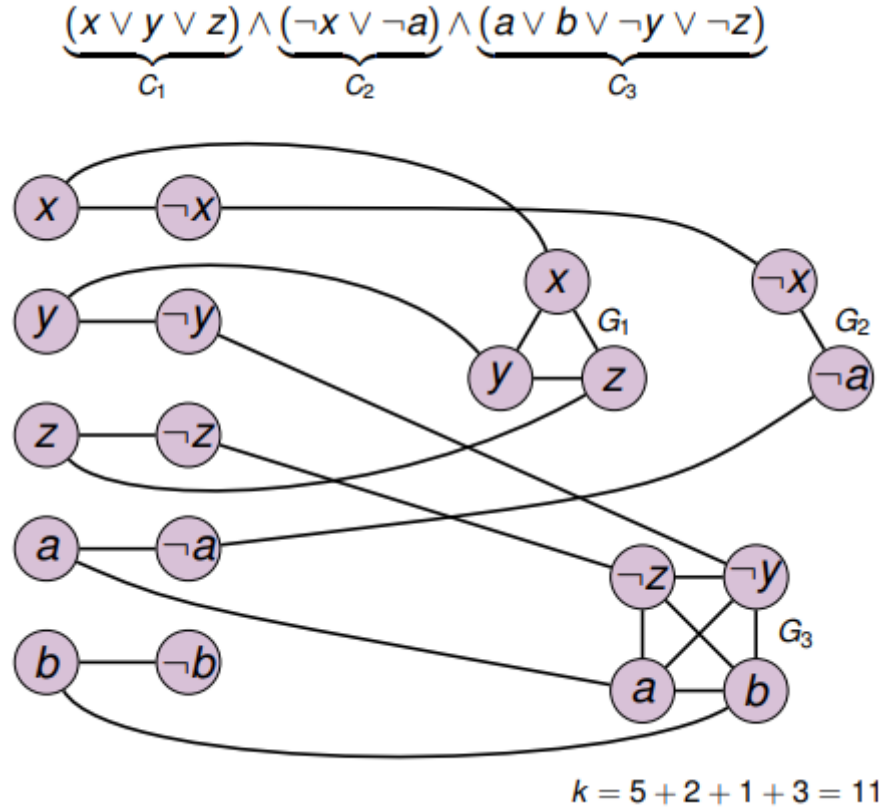
- a certificate is a proposed vertex cover

To show that Vertex Cover is NP-complete, Satisfiability (known to be **NP**-complete) can be reduced to Vertex Cover:

Given a CNF formula f with n variables and clauses $C_1, C_2, \dots, C_m$:

- for each variable x :
 - create two adjacent vertices x^t and x^f
 - these represent the literals x and $\neg x$
- for each clause C_j of size n_j :
 - create a complete graph (clique) G_j on n_j vertices
 - each vertex corresponds to one literal in the clause
- for each literal vertex in a clause gadget:
 - connect it to the corresponding literal vertex in the variable gadget
- set $k = n + \sum_{j=1}^m (n_j - 1)$

This construction can be carried out in polynomial time.



Then, exists a truth assignment that satisfies the formula f if and only if there exists a vertex cover of the constructed graph with size at most k .

Therefore, since SAT is NP-complete, it follows that vertex cover is NP-complete.

17.14 NP-completeness of Clique

Recall the clique problem:

Given a finite graph G and an integer k , does G have a clique of size k ?

A clique is a set of vertices such that every pair of distinct vertices in the set is connected by an edge.

There is a straightforward reduction from Vertex Cover to Clique:

Let $\overline{G}$ denote the complement of G , i.e. the graph obtained by:

- keeping the same vertices, and
- replacing every non-edge of G with an edge, and vice versa

Then:

- A set of vertices W is a vertex cover in G if and only if $V \setminus W$ is a clique in $\overline{G}$.

17.15 NP-completeness of Hitting Set

The Hitting Set problem is as follows:

Given a collection C of subsets of a set S and a positive integer k , does there exist a hitting set for C of size at most k ?

A hitting set is a subset $S' \subseteq S$ such that S' contains at least one element from each subset from C .

Hitting Set is in **NP**:

- the subset S' is the certificate
- it can be verified in polynomially time if S' contains an element from each subset in C

To show that Hitting Set is **NP**-complete:

- restrict it to instance with $|c| = 2$ for all $c \in C$
- i.e., every set c in C has 2 elements

Then this restricted problem is equivalent to Vertex Cover:

- let S be the set of vertices of a graph G
- let each subset $c \in C$ correspond to an edge of G
 - if an edge joins vertices u and v , then the corresponding subset is u, v

A hitting set S' must contain at least one element from every subset in C .

Equivalently:

- S' must contain at least one endpoint of every edge in G

which is exactly the definition of a vertex cover.

Therefore:

- Vertex Cover $\leq$ Hitting Set

17.15.1 NP-completeness of 3-SAT

To show that 3-SAT is NP-complete, SAT can be reduced to 3-SAT:

Replace every clause

$$C = x_1 \vee x_2 \vee \dots \vee x_k$$

with $k > 3$ by:

$$C' = (x_1 \vee x_2 \vee y_1) \wedge (\neg y_1 \vee x_3 \vee y_2) \wedge \dots \wedge (\neg y_{k-3} \vee x_{k-1} \vee x_k)$$

C is satisfiable if and only if C' is, since at least one of the literals other than the y 's must be TRUE.

17.16 Reducing SAT to Clique

Another way to show that Clique is **NP**-complete is to reduce SAT to Clique.

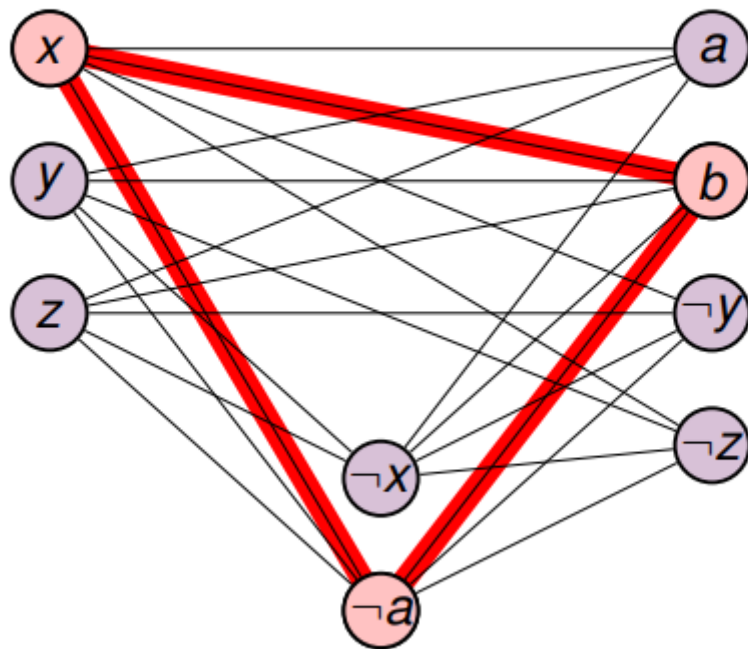
Given a CNF formula f with clauses $C_1, C_2, \dots, C_k$:

- for each literal x in clause C_j , create a vertex
 - each vertex represents a specific occurrence of a literal in a clause
- add an edge between two vertices if:
 - they come from different clauses, and
 - they are not contradictory literals (i.e. not x and $\neg x$)

A clique of size k corresponds to choosing:

- exactly one literal from each clause
- such that all chosen literals are mutually consistent

$$\underbrace{(x \vee y \vee z)}_{C_1} \wedge \underbrace{(\neg x \vee \neg a)}_{C_2} \wedge \underbrace{(a \vee b \vee \neg y \vee \neg z)}_{C_3}$$



$k = 3$

[]: